

Hardware Initialization and LVGL Binding

Hardware Initialization and LVGL Binding

1. Course Objectives
2. Core Source Code Analysis
 - 2.1 Display System Initialization (`display_init`)
 - 2.2 LVGL Graphics Library Initialization (`lvgl_init`)
 - 2.3 Core Callback: `disp_drv_flush_cb`
3. Touch Screen Driver (`touch_screen` Class)
4. Summary

1. Course Objectives

In the previous section, we saw that `main.py` called `start()`. In this lesson, we will dive deep into `ybMain/main.py` to analyze how the system "lights up" the screen and binds the LVGL graphics library with the K230's underlying hardware (display screen, touch screen).

2. Core Source Code Analysis

File Path: `src/canmv/port/builtin_py/ybMain/main.py`

2.1 Display System Initialization (`display_init`)

K230's display system is more complex than ordinary microcontrollers, involving the concept of **PipeLine (video path)** (using Pipeline is to facilitate multi-channel AI vision analysis tasks).

```
def display_init():
    global pl, config
    try:
        # ...
        display_mode = "lcd"
        display_size = [DISPLAY_WIDTH, 480]
        rgb888p_size = [640, 480]

        # Initialize Pipeline, this is the core of K230 multimedia framework
        # osd_layer_num=4 means 4 layers are reserved for screen display (On-Screen Display)
        pl = Pipeline(rgb888p_size=rgb888p_size, display_size=display_size,
                     display_mode=display_mode, osd_layer_num=4)

        # Create display channel
        pl.create(...)
    except Exception as e:
        debug_print("display_init", e)
```

Key Points:

- **PipeLine:** This is a hardware channel specifically for processing image data. The GUI does not directly write to video memory, but acts as a "layer" overlaid on video output. This allows

us to easily display camera footage in the GUI background.

2.2 LVGL Graphics Library Initialization (`lvgl_init`)

LVGL is a pure software graphics library that doesn't know how to draw pixels. We need to provide a "callback function" to tell it how to push pixels to the screen.

```
def lvgl_init():
    # 1. Initialize LVGL library
    lv.init()

    # 2. Create display driver
    disp_drv = lv.disp_create(DISPLAY_WIDTH, DISPLAY_HEIGHT)

    # 3. Set refresh callback function (most important part!)
    disp_drv.set_flush_cb(disp_drv_flush_cb)

    # 4. Allocate double buffers (Double Buffering)
    disp_img1 = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.BGRA8888)
    disp_img2 = image.Image(DISPLAY_WIDTH, DISPLAY_HEIGHT, image.BGRA8888)
    disp_drv.set_draw_buffers(disp_img1.bytearray(), disp_img2.bytearray(), ...)

    # 5. Initialize input device (touch screen)
    global_touch_dev = touch_screen()
```

2.3 Core Callback: `disp_drv_flush_cb`

This is the bridge connecting software (LVGL) and hardware (Display).

```
def disp_drv_flush_cb(disp_drv, area, color):
    # When LVGL finishes rendering part of the screen, this function is called
    # color: points to the rendered pixel data

    if disp_drv.flush_is_last() == True:
        # If this is the last piece of data for the current frame, call
        Display.show_image to display
        if disp_img1.virtaddr() == ...:
            Display.show_image(disp_img1)
        else:
            Display.show_image(disp_img2)

    # Tell LVGL that refresh is complete, ready for next frame
    disp_drv.flush_ready()
```

Working Principle Analysis:

1. LVGL draws the interface in memory.
2. Call `disp_drv_flush_cb`, passing in the drawn data.
3. We call K230's `Display.show_image()` to send the data from this memory to the display controller.
4. Using **double buffering** (`disp_img1`, `disp_img2`) can avoid screen tearing and ensure smooth UI.

3. Touch Screen Driver (`touch_screen` Class)

To make the screen respond to clicks, we need to convert physical touch coordinates to coordinates that LVGL understands.

```
class touch_screen():
    def __init__(self):
        # Create LVGL input driver
        self.indev_drv = lv.indev_create()
        self.indev_drv.set_type(lv.INDEV_TYPE.POINTER)
        self.indev_drv.set_read_cb(self.callback) # Register callback

    def callback(self, driver, data):
        # Read physical touch data
        tp = self.touch.read(1)
        if len(tp):
            # Assign physical coordinates to LVGL data structure
            data.point = lv.point_t({'x': tp[0].x, 'y': tp[0].y})
            data.state = lv.INDEV_STATE.PRESSED
        else:
            data.state = lv.INDEV_STATE.RELEASED
```

4. Summary

In this lesson, we learned about the "foundation" of the GUI system:

1. **PipeLine** is responsible for opening up the video path.
2. **LVGL** is responsible for generating graphics data.
3. **flush_cb** is responsible for moving graphics data to the video path.
4. **touch_screen** is responsible for translating finger movements into click events.

In the next section, we will explain the core part of the GUI: **AppManager**, and see how it manages those colorful application icons.